Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026	Semester:	1st
Name of the Student:	G. Chandra Rao	Roll Number:	324126552139
Name of the Lab:	Full Stack Lab	Timings:	

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva- Voce (2Marks)			
1/7	1	4	4	3	2	13		
8/7	2	4	4	4	2	14		
15/7	3	4	4	5	1	14		
22/7	4	4	3	4	1	13		
29/7	5	4	4	3	2	13		
5/8	6	4	4	3	1	12		
9/8	7	4	4	3	1	12		

Name of the Faculty:

Faculty Signature:

The relationship between HTML and CSS is often described using a metaphor. HTML is the foundation or the skeleton of a house, providing the basic structure and content of the webpage. It defines the various elements such as headings, paragraphs, images, links, tables, lists, and other multimedia content. HTML elements are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

CSS (Cascading Style Sheets) is a stylesheet language used to control the presentation and visual appearance of an HTML document. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sizes and devices. CSS can be applied using inline, internal or external stylesheets.

→ HTML and CSS ... webpages, where HTML provides the structure and content. While CSS provides the styling layout, visual presentation. The combination of both forms the foundation of modern web development and enables the creation of organized, attractive, and user-friendly web interfaces.

* Difference between MERN Stack and Full Stack

→ MERN Stack is a JavaScript-based technology stack used for developing modern web applications. MERN stands for MangoDB, Express.js, React.js, Node.js, MangoDB is used for database management, Express.js handles the backend framework and APIs. React.js is used to develop frontend user interface and Node.js provides the server-side runtime environment.

→ Full Stack Development refers to the development of both the frontend and backend of a web application along with database integration.

A Fullstack developer works with technologies used for designing the user interface, implementing server-side logic managing databases.

HTML Components :

1. HTML Tags :- Define the structure and elements of a webpage.
2. Headings :- used to display titles and headings using <h1> to <h6>
3. paragraphs :- used to display text using the <p> tag
4. Links :- used to navigate between webpages using <a> tag.
5. Images :- used to display images using <image> tag
6. Lists :- used to create ordered and unordered lists using <ol>
7. Tables :- used to organize data in rows, columns
8. Forms :- used to collect user input using elements such as text boxes buttons and check boxes
9. Div and span :- used to group and organize webpage content.
10. Semantic Elements :- Elements such as <header> <nav> <section> <article> <footer> provide meaningful webpage structure.

CSS Components

1. selectors - select HTML elements to apply styles
2. properties - Define the visual characteristic of elements
3. values - specify the setting of a CSS property
4. colors - Define text, background, border colors
5. fonts - Control font family, size, style, and weight.
6. Backgrounds - set background colors and images
7. Borders - Define the border style, width and radius.
8. Margin - controls the space outside an element.
9. padding - controls the space inside element.
10. Layout - controls the arrangement of elements using Flexbox, grid and positioning.

HTML vs HTML 5

HTML	HTML 5
older version of HTML	Latest major version of HTML
Limitted multimedia support	supports audio and video
uses external plugins for multimedia	No external plugs needed
Has fewer semantic tags	provides semantic tags
No <canvas> element	supports <canvas>
Limited web storage	supports local storage
Complex DOCTYPE declaration	simple <!DOCTYPE HTML>

CSS vs CSS 3

CSS	CSS 3
Earlier version of CSS	Advanced version of CSS
Basic styling features	Provides advanced styling features
Limited animations	supports animations and transitions
No flexible layout systems	supports Flexbox and Grid
Limited visual effects	supports shadows, gradients effects
No media queries	supports ~~shadows~~, media queries
uses a single specification	Divided into different modules

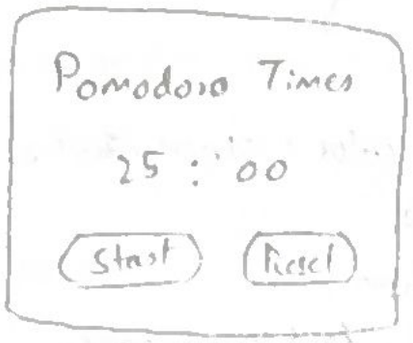
Design a static webpage using HTML and CSS

Pomodoro Timer, using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title Pomodoro Timer </title>
    <style>
        body {
            font-family: Arial;
            background: #f5f5f5;
            text-align: center;
            padding: 50px;
        }
.timer {
    width: 300px;
    margin: auto;
    padding: 30px;
    background: white;
    border-radius: 15px;
    box-shadow: 0 0 10px gray;
}
h1 {
    color: #e74c3c;
}
.time {
    font-size: 50px;
    margin: 20px;
}
```

```
    button {
        padding : 10px 20px;
        margin : 5px;
        border : none;
        border-radius : 5px;
        background : #e74c3c;
        color : white;
    }
    </style>
</head>
<body>
    <div class = "Timer">
        <h1> Pomodoro Timer </h1>
        <div class = "time"> 25:00 </div>
        <button> start </button>
        <button> Reset </button>
    </div>
</body>
</html>
```

O/P:



1 Design an HTML code using audio tags.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My Favorite Song </h2>
    <audio controls>
        <source src = "song.mp3" type = "audio/mpeg">
        Your browser does not support the audio
        tag
    </audio>
</body>
</html>
```

O/P: Displays an audio player with play, pause and volume controls.

Steps:

Create a new HTML file
Add the basic HTML structure
Add heading using <h2> tag
Insert the <audio> tag with control attribute
specify the audio file using <source> tag
save and open the HTML file in a web browser
play the audio using displayed controls.

O/P:

Displays an audio player with play, pause and volume controls.



11. Design a HTML code using video tags.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> My Video </h2>
    <video width = "400" controls>
        <source src = " video.mp4" type = " video/mp4">
    </video>
</body>
</html>
```

O/P: Displays a video player with play, pause, volume and Full screen controls.

Steps:

Create a new HTML file
Add a basic HTML structure
Add a heading using the <h2> tag
Insert the <video> tag with controls attribute
Specify the video file using < source > tag
save the file and open it in web browser
play the video using displayed controls.

Output :

Displays an ~~audio~~ video player with play, pause and volume controls

- My Favourite Video



03 Write a program using Java Script (arrays, functions)

code :

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];

function displayArray(arr){
    console.log("Array Elements:");
    for (let i=0; i<arr.length; i++){
      console.log(arr[i]);
    }
}
function findSum(arr){
    let sum=0;
    for (let i=0; i<arr.length; i++){
        sum += arr[i];
    }
    return sum;
}
```

O/P

```
12
13
14
15
16
19
24
34
67
90
99
```

Sum: 403

Experiment - 04

4. Develop dynamic webpage using java script

Program:

```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic webpage </title>
    <style>
        body {
            text-align: center;
            font-family: Arial;
            margin-top: 100px;
        }
        button{
            padding: 10px 20px;
            font-size: 16px;
            cursor: pointer;
        }
    </style>
</head>
</body>
    <h1 id = "heading"> welcome to My
                        webpage </h1>
    <p id = "message"> click the button
                to change this webpage. </p>
    <button onclick = "change page()">clickme
                </button>
    <script>
        function changepage() {
            document.getElementById("heading").
            innerHTML = "welcome to javascript!";
            document.getElementById("heading").
            innerHTML = "The webpage has been
                changed dynamically";
            document.body.style.backgroundcolor = "light
        }
    </script>
</body>
</html>
```

Output:

welcome to My Webpage

click the button to change this webpage

[click Me]

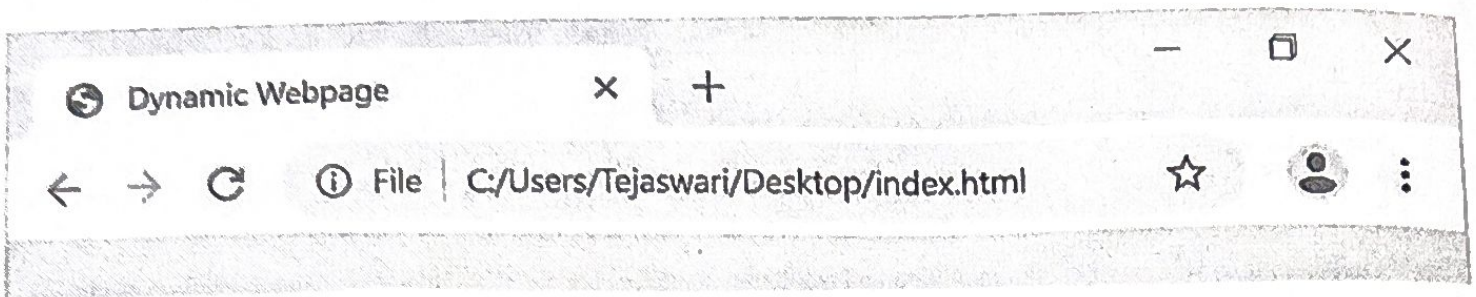
→ When the user clicks "Click Me", the javascript function changepage() is called, then the heading dynamically changes and background also changes

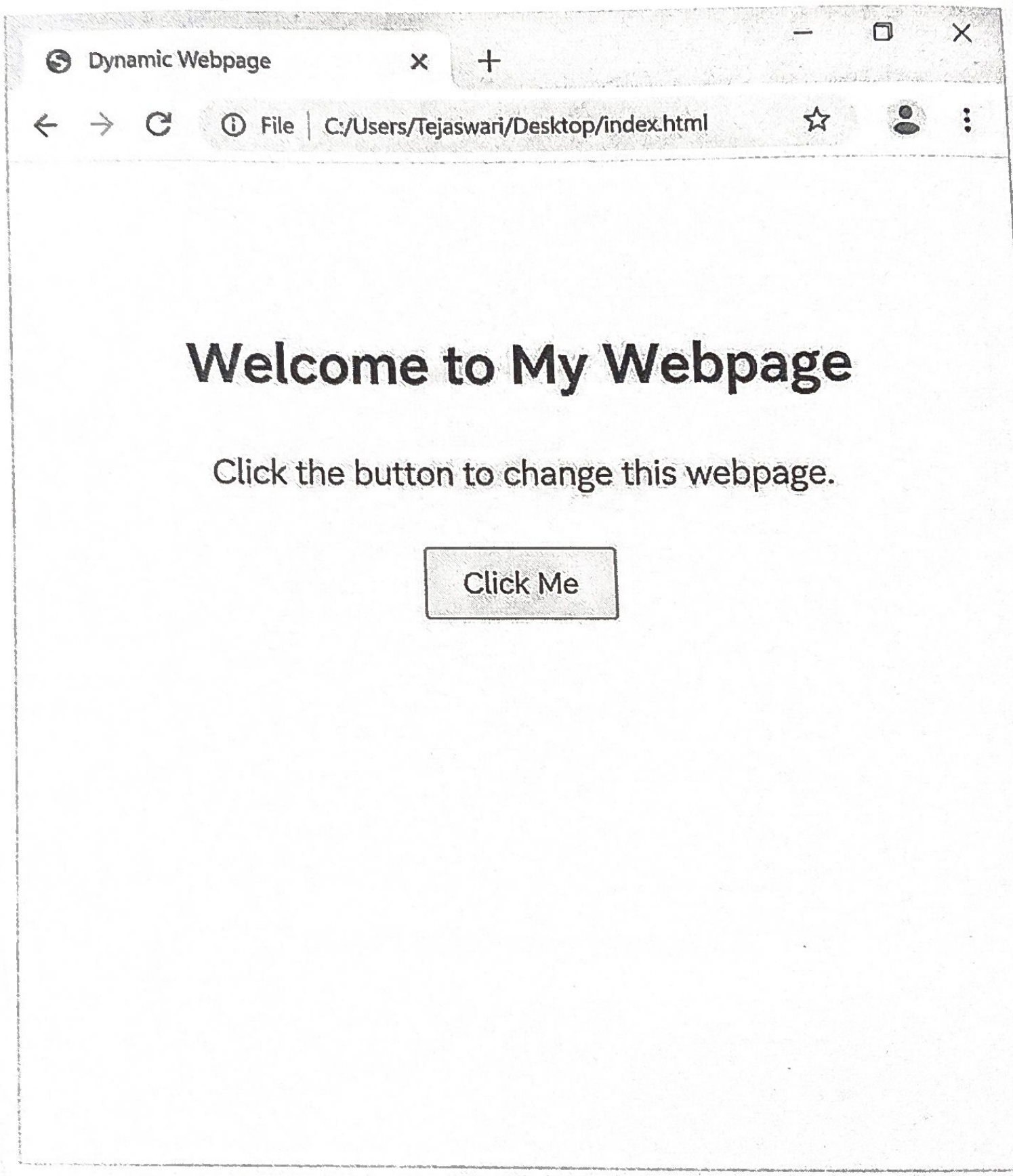
- After clicking the button:

welcome to Javascript

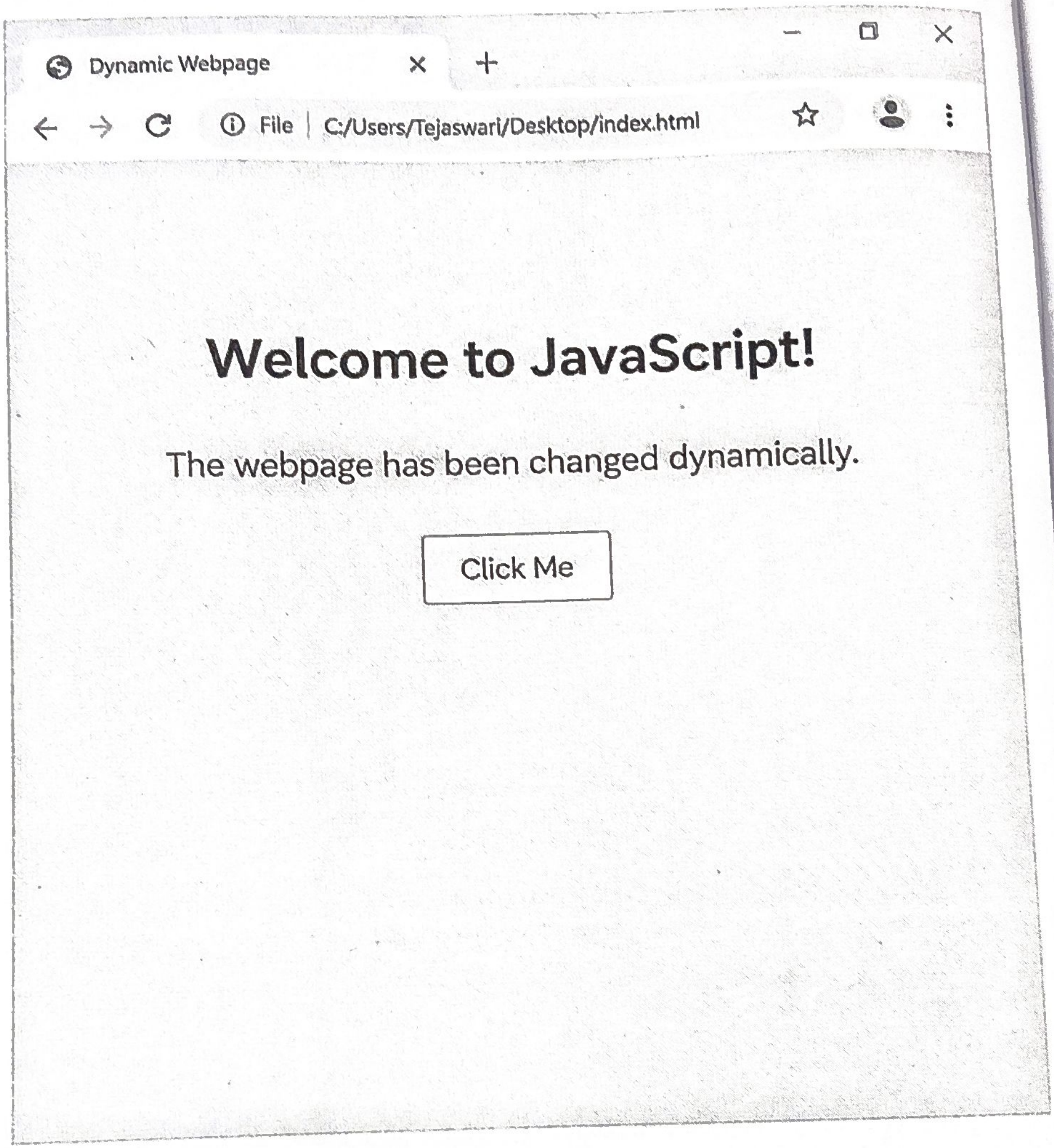
The webpage has been changed dynamically

[clickme]





Before clicking the button



After clicking the button

ri/Desktop/index.html

i/Desktop/index.html

My Webpage

nge this webpage.

utton

Experiment - 5

Implement Modules in Node JS

In Node.js you can implement modules using distinct systems: Common JS (CJS), which is traditional default and ES modules (ESM), the modern java script standard.

1. The common JS method (default):

Common JS uses module exports to share code and require() to import it. This system loads modules synchornously.

a. Create the module (math.js):

```
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;
module.exports = {
    add,
    subtract
};
```

b. use the module (app.js)

```
const math = require('.\math');
Console.log(math.add(10,5));
Console.log(math.subtract(10,5));
```

O/P: 15
5

2. The ES modules method (modern standard):

ES modules use export to share code and import to use it to make node.js recognize ES modules you must either use the .mjs file extension or add "type": "module" to your package.json file.

a. Create the module (calculator.js):

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculatorName = "super calc";
export default calculatorName;
```

b. use the module (main.js)

```
import calculatorName, { multiply, divide }
    from './calculator.js';
console.log(calculatorName);
console.log(multiply(4,2));
console.log(divide(4,2));
```

Output:

```
super calc
8
2
```

3. using Built-in core modules

Nodejs also comes equipped with dozens of built-in core modules (like for file systems or path for file routing) that don't need installation we will load them by module names:

```
// commonJS style
const fs = require('fs');
// FS modules style
import fs from 'fs';
```

Here are some of the most commonly used ones:

1. fs = file system operations
2. https = HTTPS server and client
3. os = operating system utilities
4. path = file path utilities
5. events = Event handling
6. util = utility functions
7. stream = stream handling
8. crypto = cryptographic functions
9. url = URL parsing
10. querystring = URL query string handling.

Example: simple HTTPS server

```
http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type':
        'text/html'});
```

```
    res.end ("Helloworld!");
});.listen(8080);
```

O/P:

Hello world!

Experiment- 06

Develop mini application using expxes and NodeJS
simple mini application using node JS expressJS

Student Managment System

1. Add a student
2. View all students
3. delete a student
4. uses Express.js router
5. Simple HTML interfaces

project Structure

```
Student.app/
    ├── server.js
    └── public
        └── index.html
```

server.js

```
Const express = require ("Express");
Const app = Express();
Const PORT = 3000;
let Students = [];
app.use (Express.json());
app.use( Express.static ("public"));
app.get ("/Student", (req, res) => {
                    res.json (Students);
        });
// Add
app.post ("/Students", (req.res) => {
                    Const student = {
```

```
    id : student.length + 1,
    name : req.body.name,
    course : req.body.course
  };
  students.push(student);
  res.json({ message : "Student added successfully",
             student:student });
});

// Delete
app.delete("/students/:id", (req, res)=>{
    const id = parseInt(req.params.id);
    students = students.filter(student =>
        student.id !== id);
    res.json({
      message : "Student deleted successfully"})
    });

// Start server
app.listen(PORT, ()=>{
    console.log("server running at http://localhost
    :${PORT}`);
});
```

Public / index.html:

```
<!DOCTYPE html>
<html>
<head>
</title> Student Management </title>
</head>
<body>
  <h1> Student Management System </h1>
  <input type = "text" id = "name"
    placeholder = "Enter student name" >
  <input type = "text" id = "course"
    placeholder = "Enter course" >
  <button onclick = "addStudent()" > Add Student
                                    </button>
  <h2> Student List </h2>
  <ul id = "studentList"> </ul>
  <script>
    function addStudent() {
let name = document.getElementById("name").
                        value;
let course = document.getElementById("course").
                        value;
fetch("/students", {
        method: "post",
        headers: {
          "Content-TYPE": "application/json"},
        body: JSON.stringify({
            name: name,
            course: course
        })
    })
.then(response => response.json())
.then(data => alert(data.message));
```

```
document.getElementById("name").value = "";
document.getElementById("course").value = "";
        displayStudents();
    });
}

function displayStudents() {
    fetch("/students").then(response => response.json())
    .then(students => {
        let list = document.getElementById("studentList");
        list.innerHTML = "";
        students.forEach(student => {
            let li = document.createElement("li");
            li.innerHTML = student.id + " - " + student.name + " - " + student.course + " <button onclick='deleteStudent(" + student.id + ")'>Delete</button>";
            list.appendChild(li);
        });
    });
}
function deleteStudent(id) {
    fetch("/students/" + id, { method: "DELETE" })
    .then(response => response.json())
    .then(data => {
        alert(data.message);
        displayStudents();
    });
}
```

```
displayStudents();
</script>
</body>
</html>
```

Output:

Student Management System

Enter student name

Enter course

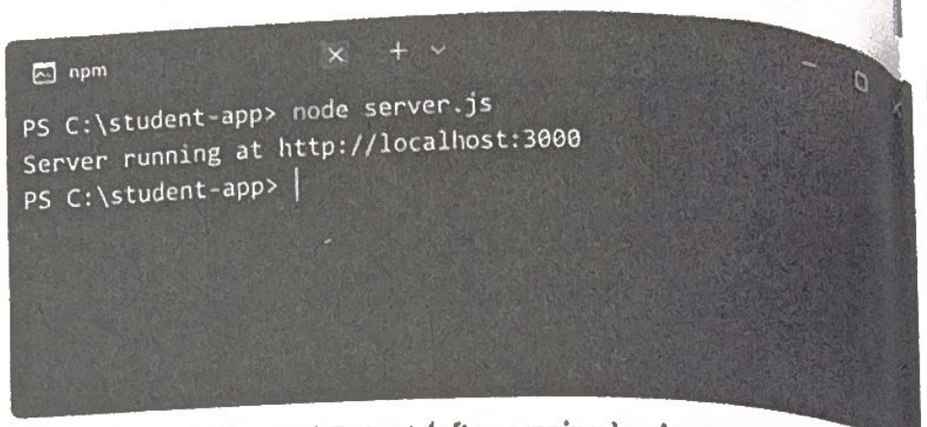
Add student

Student list:

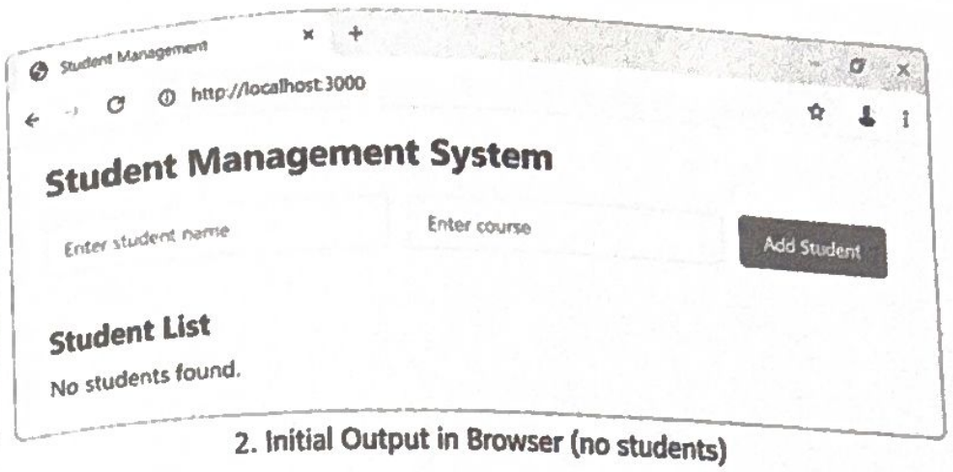
1. Ram - CSM [delete]
2. Sita - CSE [delete]

The Express.js server is successfully started at http://localhost:3000

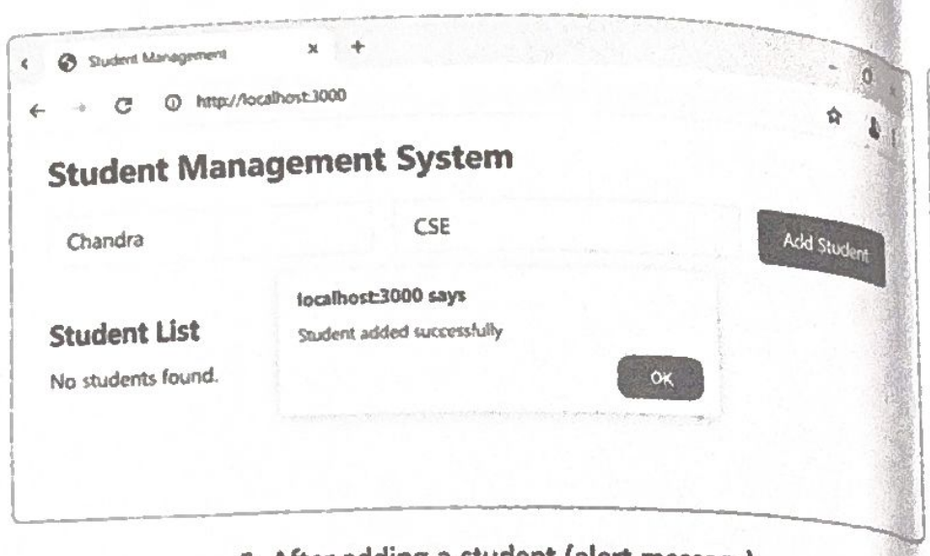
The browser displays the student management system with fields.



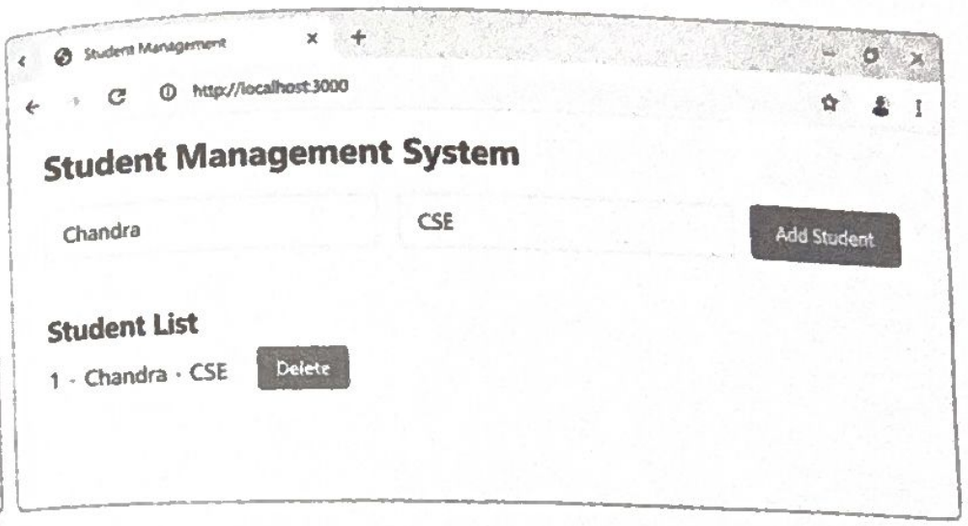
1. Terminal Output (after running `node server.js`)



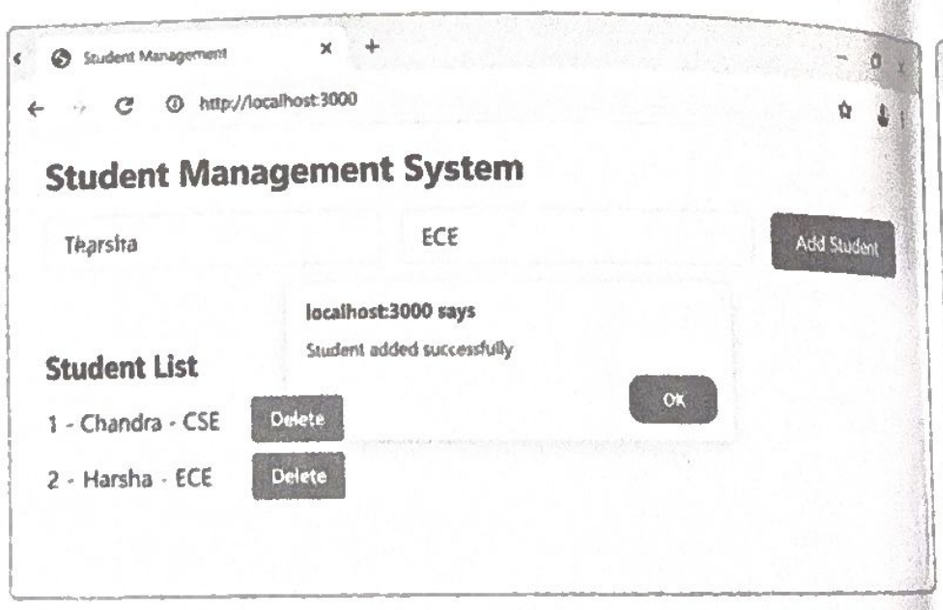
2. Initial Output in Browser (no students)



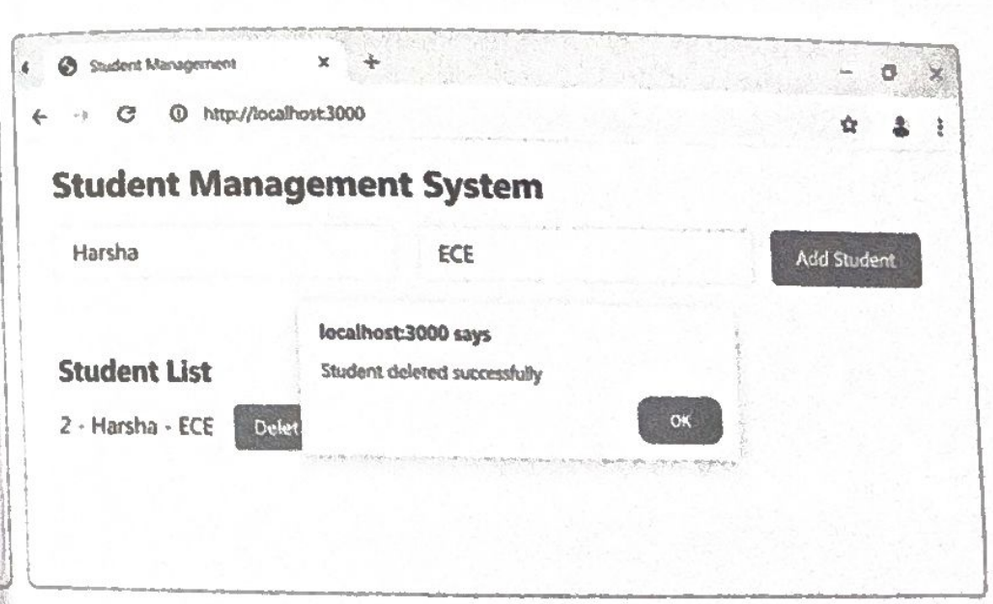
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment - 04

Create a functional component and class component using react.

Aim: To develop a react application using Function components and class components to display student details. manage state. handle button clicks and dynamically update data on a webpage.

Functional Component:

A Functional component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as use state to manage state and update the webpage dynamically.

program:

```
import React, {use state} from
'react';
function student functional (){
    const [name, setName] = use State ("Chandu")
    const [course, set Course] = use state ("CSM")
    const [ count, set Count) = use state (0);
    const diange Name = () => {
        setName ("Raj");
    };
    const change Course = () => {
        set Course ("CSD")
    };
    const increment Count = () => {
        set (count + 1);
    };
return (
    <div style = {{
        border : "2px solid blue";
        Padding: "20 px",
        margin : "20 px",
        border Radius: "10 px";
        text Align : "center"
    }}>
<h1> Student Details </h1>
 <h2> Functional Component </h2>
 <P> <b> Name : </b> {name} </P>
 <P> <b> COurse: </b> {course} </P>
 <P> <b> Button clicks : </b> {count} </P>
 <button on click = {change Name}>
        change Name </button>
 <button onclick = {change Course} change
        course </ buttons>
 <button onclick = {increment Count}> Increment
        </button>
    </div>
);
```

Output:

when the user clicks the button, the name and count changes dynamically without reloading the webpage.

class component:

A class component is an ES6 javascript class that extends React.Component It uses the render() method to return JSX and manages state using this.state and this.setState().

program.

```
import React {Component} from
'react';
class Student(class extends Component {
        constructor(props) {
            super(props);
            this.state = {
                name : "Chandra"
                course : "Computer Science"
                count : 0
            };
        }
    changeName = () => {
        this.setState({
            name: "Rao"
        });
    };
    changeCourse = () => {
        this.setState({
            course: "CSM"
        });
    };
```

increment count = () => {

this.setState((prevState) => ({

React App

localhost:3000

React Components

Student Details

Functional Component

Name: Chandra

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Student Details

Class Component

Name: Chandra

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Experiment - 08

5) Implement http methods using JSON server

Aim: To implement HTTP methods such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD (creat, Read, updat and Delete) operations on student xacts.

Theory: JSON server is a tool used to create a fake REST API using JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to communicate b/w a client and a server.

* GET - Retrieves data from the server
* POST - Adds new data to the server
* PUT - Replaces an existing record
* PATCH - updates selected fields of a record
* DELETE - Removes a record from the server

a) Creat a JSON server:

Step 1: Initialize the project

open the VS code Terminal and execute the following Commands:

```
mkdir JSON server
cd. JSON server
npm init-y
npm install json server
```

Step 2: Create db-json

create a file named db.json and add the following student data

```
{ "students": [
    { "id": "1"
      "name": "Chandra",
      "course": "CSM"
      "marks": 90
    },
    { "id": "2"
      "name": "Rao",
      "course": "CSM",
      "marks": 90
    }
    {"id": "3"
     "name": "Ram"
     "Course": "IT"
     "marks": 88
```

Step 3: Start the JSON server

for the current JSON server version, execute:

```
npm json.server db.json
```

The server start at:

```
http://localhost:3000
```

The student records can be accessed using

```
http://localhost:3000/students.
```

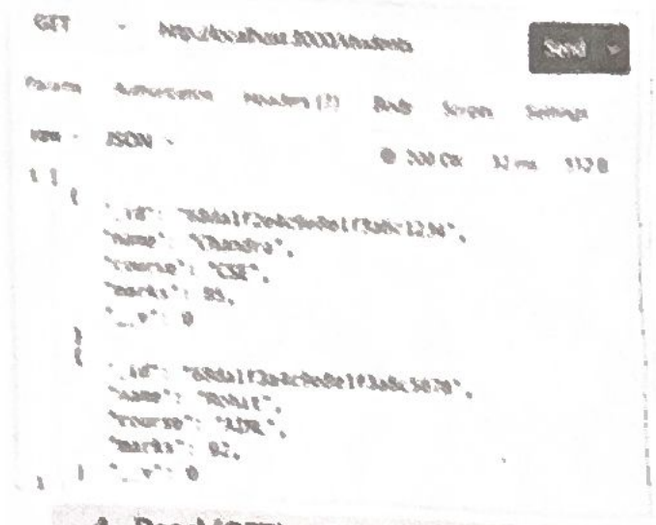


: Implementation of HTTP methods :

a) GET Method: The GET method is used to retrieve student records from JSON server.

Code:

```
fetch('http://localhost:3000/students')
.then(response => response.json())
.then(data => {
        console.log(data);
    })
.catch(error => {
        console.log(error);
    });
```

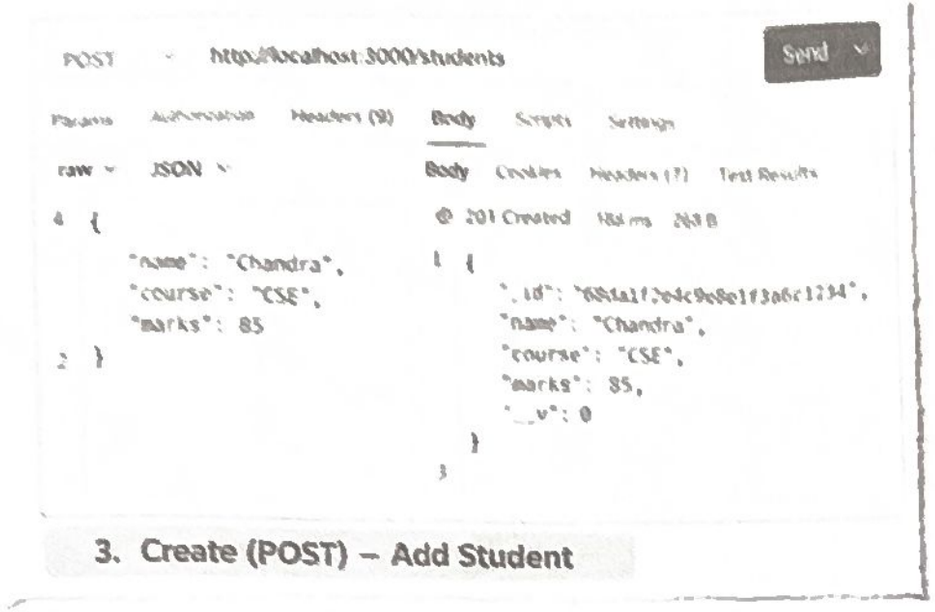


4. Read (GET) – Get All Students

b) POST method: The POST method is used to insert a new student record into the server.

Code: fetch('http://localhost:3000/students', {

```
    method: 'POST',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'Rahul',
        course: 'CSE',
        mark: 92
    })
})
.then(response => response.json())
.then(data => {
        console.log(data);
})
.catch(error => {
        console.log(error);
```



3. Create (POST) – Add Student

c) PUT method: The PUT method is used to replace an existing student record with updated data

Code: fetch ('http://localhost:3000/students/1', {

method: 'PUT',

headers: {

'Content-Type': 'application/json'

},

body: JSON.stringify({

name: 'Chandra',

course: 'AIML'

marks: 95

})

})

.then (response => response.json())

.then (data => {

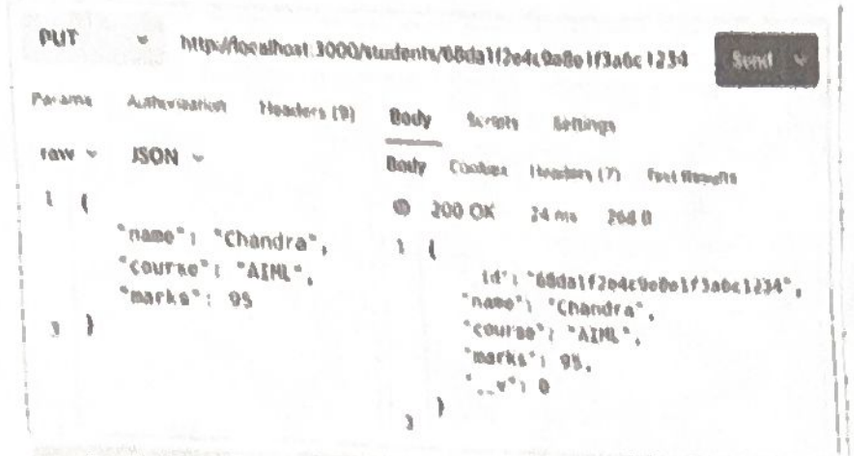
console.log(data);

})

.catch (error => {

console.log(error);

});



6. Update (PUT) – Update Student

d) PATCH Method: The PATCH method is used to update only selected fields of an existing student records.

Code: fetch ('http://localhost:3000/students/2', {

method: 'PATCH',

headers: {

'Content-TYPE': 'application/json'

}

body: JSON.stringify({

marks: 96

})

})

.then (response => response.json())

.then (data => {

console.log(data);

})

.catch (error => {

console.log(error);

});

GET http://localhost:3000/students/68da1f2e4c9e8e1f3a6c1234 Send

Params Authorization Headers (7) Body Scripts Settings

raw JSON 200 OK 28 ms 246 B


```
{
    "_id": "68da1f2e4c9e8e1f3a6c1234",
    "name": "Chandra",
    "course": "CSE",
    "marks": 85,
    "__v": 0
}
```


5. Read (GET) – Get Student by ID

E) DELETE method: The DELETE method is used to remove an existing student record from the server

Code:

```
fetch ('http://localhost:3000/students/', {
    method: 'DELETE'
})
.then (response => {
    if (response.ok) {
        console.log
```

DELETE http://localhost:3000/students/68da1f2e4c9e8e1f3a6c1234 Send

Params Authorization Headers (7) Body Scripts Settings

Body Authorization Headers (10)

200 OK 20 ms 210 B


```
{
    "message": "Student deleted successfully"
}
```


7. Delete (DELETE) – Remove Student

Experiment - 09

4. Develop CRUD operations using MongoDB

Aim: To develop a webapplication using Node.js Express.js and MongoDB to perform CRUD operations (create, read, update, delete) on student records.

Theory: MongoDB is a NoSQL database that stores data in flexible, JSON like documents. CRUD operations are used to manage data in a database.

Mongoose in an object Data Modeling (ODM) library for MongoDB and Nodejs. It helps define schemas, interact with collections and perform database operations.

1) **Installation and Project Setup:**

```
mkdir MongoDB - CRUD
cd MongoDB - CRUD
npm init -y
npm install express mongoose
```

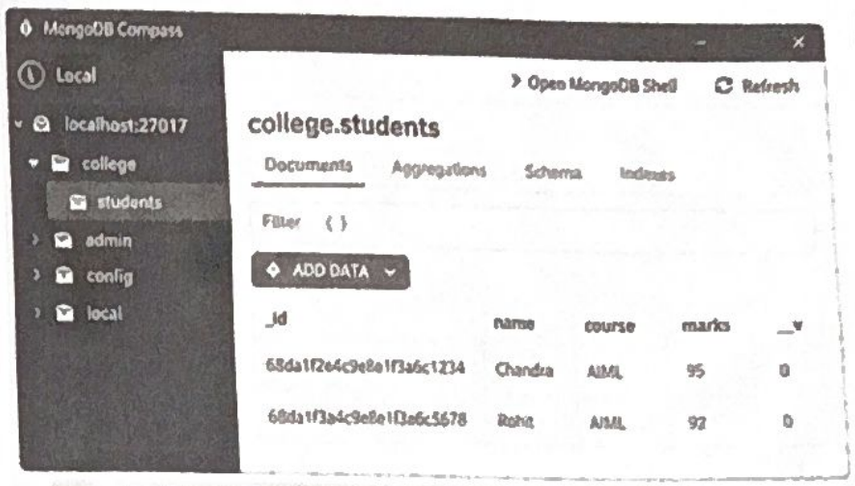
2) **Program:** server.js

```
Const express = require ('express');
const mongoose = require("mongose");
const app = express();
const PORT = 3000;
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:2701
                  /college')
.then(() => console.log("MongoDB Collected"))
.catch(err => console.log(err));
// create student
const studentschema = new
    mongoose.schema({
        name: {
            type: string,
            required: true
        },
        Course: {
            type: string,
            required: true
        }
        marks: {
            type: Number,
            required: true
        }
    });
// create student model
Const student =
    mongoose.model('student', studentschema
// create : Add
app.post('/students', asgn(req, res) => {
    try {
        Const student = new student(req.body);
```

```
const saved student = ...
res.status(201).json(savedStudent);
} catch(error){
    res.status(400).json({message:
        error.message});
    }
});
// Read : Get all students
app.get('/students', async(req, res) => {
  try{
    const students = await
    student.find();
    res.status(500).json({message: error,
                          message});
    }
});
// READ: Get student by ID
app.get('/students/:id', async(req,res) =>{
  try {
    const student = await
    student.findById(req.params.id);
    if(!student){
      return res.status(404).json({
        message: "Student not found"
      });
    }
    res.status(200).json(student);
  } catch(error){
    res.status(400).json({message: error.message});
  }
});
// UPDATE
app.put('/students/:id', async(req, res)=>{
  try{
    const student = await
    Student.findByIdAndUpdate(
      req.params.id,
      req.body
      {new: true, runValidators: true}
    );
    if(!student){
      return res.status(404).json({
        message: "Student not found",
      });
    }
    res.status(200).json(student);
  } catch(error){
    res.status(400).json({message: error.message});
  }
};
// DELETE
app.delete('/students/:id', async(req, res)=>{
  try{
```

```
const student = await Student
    .findByIdAndDelete(
        req.params.id
    );
if (!student) {
    return res.status(404).json({
        message: "student not found"
    });
}
res.status(200).json({
    message: "student deleted successfully"
});
} catch (error) {
    res.status(400).json({message: error.message})
}
});
```





8. MongoDB Compass – Database View